

Instituto por Diversificado por Cooperativa de Villa Hermosa

Materia – Programación

Docente- Daniela

Tema – Video Juego 1ra. Parte

Nombre – Sheily Alejandra Charuc Revolorio

Clave – 5

Grado – 5to. Perito Contador

Sección – C

Video Juego

Idea

El juego trata sobre un superhéroe aventurero al que un poderoso hechicero le arrebató sus habilidades y secuestra a su madre. Para rescatarla, deberá emprender un peligroso viaje a través de diferentes escenarios, enfrentándose a criaturas y superando diversos desafíos.

Durante su aventura, el jugador deberá explorar el mundo, recolectar recursos, obtener armas y herramientas, formar un equipo con personajes que encontrará en el camino y recuperar sus poderes poco a poco. Cada decisión influirá en la forma en que avance la historia hasta llegar al enfrentamiento final contra el hechicero y liberar a su madre.

Se usará:

Manejo de errores, Tuplas, listas, funciones, ámbito de variables

El código será:

Código modular y fácil de entender.

Objetivo del juego:

Desarrollar un juego organizado, modular y legible, donde el jugador deba explorar, sobrevivir y enfrentarse a diferentes enemigos hasta rescatar a su madre y recuperar todos sus poderes.

Análisis:

Usuario: estudiantes 16-19

Tipo de video juego: Aventura, supervivencia y batallas

Aspectos

Divertidos – El jugador explorará diferentes escenarios, resolverá pequeños desafíos, encontrará personajes que podrán ayudarlo y obtendrá recompensas conforme avance en la historia.

Competencia – Deberá enfrentarse a enemigos cada vez más fuertes, administrando correctamente sus recursos, armas y habilidades para poder continuar.

Enemigo principal – Un hechicero oscuro que robó los poderes del protagonista y secuestró a su madre. Además, enviará monstruos y guardianes para impedir que el jugador llegue hasta él.

Supervivencia – El jugador deberá mantener su salud, buscar comida, agua y refugio, además de fabricar herramientas y armas para sobrevivir.

Premio – Al completar la aventura, el protagonista rescatará a su madre, recuperará todos sus poderes y desbloqueará un final especial.

niveles (minijuegos) - **Nivel 1 - El bosque perdido**

- Buscar comida.
- Fabricar una espada de madera.
- Derrotar al primer enemigo.

Nivel 2 - La cueva oscura

- Resolver un pequeño acertijo.
- Encontrar minerales.
- Mejorar las armas.

Nivel 3 - La aldea olvidada

- Hablar con diferentes personajes.
- Reclutar un aliado.
- Comprar objetos con monedas obtenidas.

Nivel 4 - El castillo del hechicero

- Derrotar a los guardianes.
- Recuperar los poderes perdidos.
- Enfrentarse al jefe final.

Herramientas – El jugador podrá obtener:

- Espada.
- Escudo.
- Arco.
- Flechas.
- Pociones de vida.
- Comida.
- Agua.
- Madera.
- Piedra.
- Hierro.
- Llaves especiales.

- Monedas.

Requerimientos

Funcionales

Son las acciones, comportamientos y características que el sistema debe ejecutar obligatoriamente para que el jugador pueda interactuar con el juego.

1. Gestión del Progreso e Historia (Niveles y Estado)

- El sistema debe permitir al usuario avanzar de forma lineal a través de 4 niveles específicos (Bosque perdido, Cueva oscura, Aldea olvidada y Castillo del hechicero).
- El juego debe registrar el estado actual del héroe (salud, poderes recuperados, ubicación).
- El sistema debe habilitar el "enfrentamiento final" y el "final especial" solo cuando se completen las condiciones del Nivel 4.

2. Sistema de Supervivencia y Estadísticas

- El sistema debe controlar y actualizar las estadísticas de supervivencia del jugador: Salud, Hambre (comida) y Sed (agua).
- El juego debe restar puntos de supervivencia con el paso del tiempo o tras ciertas acciones, y sumar puntos al consumir alimentos o agua.

3. Sistema de Inventario y Recursos

- El sistema debe almacenar y gestionar los recursos del jugador (Madera, Piedra, Hierro, Monedas, Llaves especiales) utilizando **listas o tuplas**.
- El jugador debe poder recolectar, comprar y mejorar herramientas/armas (Espada, Escudo, Arco, Flechas, Pociones de vida).

4. Interacciones y Mecánicas por Nivel

- **Nivel 1:** El sistema debe validar la recolección de comida, permitir la fabricación de la espada de madera y procesar el combate contra el primer enemigo.
- **Nivel 2:** El sistema debe presentar un acertijo con validación de respuesta correcta, permitir la recolección de minerales y la mejora de armas en el inventario.
- **Nivel 3:** El juego debe desplegar diálogos con personajes no jugables (NPCs), permitir el reclutamiento de un aliado y habilitar una tienda para comprar objetos usando monedas. [1]
- **Nivel 4:** El sistema debe procesar combates sucesivos (guardianes y jefe final) y desbloquear los poderes del protagonista.

5. Sistema de Combate y Decisiones

- El sistema debe calcular el daño infligido y recibido durante los enfrentamientos con enemigos y jefes basándose en las armas equipadas.
- El juego debe ofrecer opciones de decisión al jugador que alteren el flujo de la narrativa o el orden en que se consiguen los objetivos.

No funcionales

Son las propiedades, cualidades y restricciones técnicas que debe cumplir el software, enfocadas en cómo se construye y cómo opera el programa.

1. Restricciones de Código y Arquitectura (Ámbito Académico)

- **Modularidad:** El código fuente debe estar obligatoriamente dividido en funciones específicas (por ejemplo: `inicializar_juego()`, `combate()`, `gestionar_inventario()`) para garantizar que sea fácil de entender.
- **Uso de Estructuras de Datos:** El programa debe implementar **listas** (para elementos mutables como el inventario o aliados) y **tuplas** (para datos constantes como las estadísticas base de los enemigos o nombres de los niveles).
- **Ámbito de Variables:** Se debe respetar el alcance de las variables locales y globales, evitando el uso excesivo de variables globales y pasando parámetros de forma correcta entre funciones.

2. Robustez y Tolerancia a Fallos

Manejo de Errores: El sistema debe incluir bloques de control de excepciones (como `try-except` si se programa en Python) para evitar que el juego se cierre inesperadamente ante entradas inválidas del usuario (por ejemplo, si el usuario ingresa texto cuando se le pide un número).

3. Usabilidad y Atractivo (Público Objetivo)

Legibilidad de Interfaz: Dado que está dirigido a estudiantes de 16-19 años y parece ser un juego de texto o interfaz gráfica simple, los menús de opciones y las descripciones de los escenarios deben ser claros, dinámicos y fáciles de leer.

- **Flujo Narrativo:** Las transiciones entre la supervivencia, el combate y los minijuegos deben ser fluidas y mantener el interés del jugador.

4. Mantenibilidad

El código debe incluir comentarios pertinentes y una nomenclatura clara de variables y funciones (CamelCase o snake_case consistente) para que cualquier otro estudiante o profesor pueda leerlo, corregirlo o expandirlo rápidamente.

